The luxd Listen-Leg Lifecycle: a Z Specification

Formal model of listener ownership and callback registration

July 2026

1 Introduction

A client holds one persistent WebSocket to `luxd`. Over that leg it receives the pub-sub events it subscribed to and the menu-callback clicks routed to its session. The connection is keyed by an *identity-derived* connection id, so two successive sessions of the same identity — an old one dying and a new one reconnecting after a 100ms backoff — address one and the same slot in the Hub's registries.

That sharing is the whole difficulty. Each session installs itself as the connection's listener and writer on connect, and removes them again on teardown; but a session that has already lost its socket may still be *suspended* inside its `finally` block, awaiting a cancelled writer task, while its successor completes an entire connect. When the old session finally resumes, an unconditional teardown removes the *successor's* state. The successor keeps its lease alive with keepalives, so the damage does not heal: a session that is live, that believes it is push-reachable, and that owns no menu entries.

A second, independent gap lies in registration. A menu callback may be registered only by a connection that holds a listen leg, because a menu item that cannot be pushed to cannot launch at the speed a menu implies. That gate reads the router under the router's lock and then writes the registry under the registry's lock. The two are not one critical section, so a teardown may pass between them and leave a callback registered with no listener — and, since the withdrawal has already happened, with nothing that will ever withdraw it.

This document models the leg as a finite state machine over a small fixed set of sessions and callbacks, so that ProB can enumerate every interleaving and either confirm the four safety invariants or produce the exact offending trace.

Concurrency model. Every session action runs on `luxd`'s single asyncio loop, so a run of code between two `await`s is atomic with respect to other session actions. `run`'s prologue (`record`, `register_writer`, `add_listener`) is one such run, and so is `_teardown`, which contains no `await` at all. Registration and click routing arrive from *other* threads — an MCP tool thread, a REST worker, the click-dispatch thread — and are therefore free to interleave anywhere, including between two of `_teardown`'s writes. The model encodes this asymmetry with a single component, `looprun`, which is non-empty exactly while a loop-side run is in progress and which guards the loop-side operations and only those.

2 Basic Types

$[SESSION, CALLBACK]$

`SESSION` is the set of `HubListenSession` instances that may occupy the one connection over time. A session object is constructed afresh by each run of the `/ws` route and installs itself exactly once, so its identity is a monotone stamp: no live incarnation ever reuses the identity of an earlier one. That property is what lets ownership be decided by comparing a stored listener to `self`, with no generation counter. Two sessions suffice to exhibit both defects; the carrier is bounded by ProB's `DEFAULT_SETSIZE`.

`CALLBACK` is the set of menu callbacks the connection's app may register. Two are enough to let a stale registration and a fresh one coexist.

3 Free Types

$SPHASE ::=$
$\quad sfresh \mid sattached \mid spumping \mid ssuspended \mid stearing \mid sdone$

One session's own progress. `sfresh`: constructed, nothing installed. `sattached`: the prologue has run — the registry entry is recorded and the writer and listener are installed — and the session is suspended on the handshake write. `spumping`: the handshake landed and the read loop is running. `ssuspended`: the read loop has ended (peer gone, protocol error, or a failed handshake write) and the coroutine is suspended in `_pump`'s `finally`, awaiting the cancelled writer task — the window this model exists to reason about. `stearing`: the teardown's detaching step has fired and the disconnect cascade is still to run. `sdone`: finished.

$$FLAG ::= leaseon \mid leaseoff$$

Whether the connection's registry entry is still within its lease. Any recorded contact renews it; it lapses only when no session is attached, and the sweep that follows takes the whole registry entry — its callbacks, its hold, and its listener slot. What the sweep cannot take is the writer, which is bound in the Hub rather than on the entry.

4 State

The state is flat: one schema, ten components. Its predicate carries only *structural* constraints — cardinality bounds on the single-occupancy slots, and the disjointness of pending and committed registrations. These hold in every design, correct or buggy, so they act as coherence constraints, not as the safety properties under test. The safety properties (Section 10) are deliberately *absent* here, so that a violating state remains reachable and ProB can find it.

Leg

$$\begin{aligned}
&phase : SESSION \longrightarrow SPHASE \\
&listener : \mathbb{P}\, SESSION \\
&writer : \mathbb{P}\, SESSION \\
&incumbent : \mathbb{P}\, SESSION \\
&owner : CALLBACK \pfun SESSION \\
&gated : CALLBACK \pfun SESSION \\
&held : \mathbb{P}\, CALLBACK \\
&leased : FLAG \\
&looprun : \mathbb{P}\, SESSION \\
&clobbered : \mathbb{P}\, SESSION \\
\hline
&\#listener \leq 1 \\
&\#writer \leq 1 \\
&\#incumbent \leq 1 \\
&\#looprun \leq 1 \\
&\mathrm{dom}\, gated \cap \mathrm{dom}\, owner = \varnothing
\end{aligned}$$

`listener` is the router's registered `CallbackListener` for this connection, modelled as a set of size at most one so that the empty set stands for "none" without a sentinel; `writer` is the Hub's registered pub-sub writer, held the same way. `owner` maps each *registered* callback to the session that installed it, so dom *owner* is the registry entry's callback set and ran *owner* its provenance; `gated` is the same for registrations that have passed the push-reachability gate but have not yet been written. `held` is the router's hold — the clicks buffered for this connection.

Two components are ghosts, present only to state properties of the transition relation as properties of a state. `incumbent` names the session whose connect most recently completed and which has not itself detached: the session that a correct design keeps push-reachable. `clobbered` accumulates any session that, while tearing down, removed state another session had installed.

`looprun` is the loop's run-to-completion token, as described in the introduction.

5 Initialization

Init

Leg'

$phase' = (\lambda\, s : SESSION \bullet sfresh)$
$listener' = \varnothing$
$writer' = \varnothing$
$incumbent' = \varnothing$
$owner' = \varnothing$
$gated' = \varnothing$
$held' = \varnothing$
$leased' = leaseoff$
$looprun' = \varnothing$
$clobbered' = \varnothing$

A single initial state: nothing connected, nothing registered, no lease.

6 The Connect Path

`Connect` is `run`'s prologue, which the loop executes as one uninterrupted run between the `accept` await and the handshake write: the registry entry is recorded (renewing the lease), the pub-sub writer is bound, and the session installs itself as the connection's listener, displacing whatever listener was there.

Installing a listener also clears the connection's callbacks. This is the rule that makes the listener slot the single owner of the connection's callback set: a callback may exist only while the session that registered it holds the slot, so a new occupant starts empty and the app re-registers from its `on_connect`. Without this clause a superseded session's entries outlive it with no one left to withdraw them (Section 13, variant 3).

Connect

ΔLeg
$s? : SESSION$

$looprun = \varnothing$
$phase\ s? = sfresh$
$phase' = phase \oplus \{s? \mapsto sattached\}$
$listener' = \{s?\}$
$writer' = \{s?\}$
$incumbent' = \{s?\}$
$owner' = \varnothing$
$leased' = leaseon$
$gated' = gated$
$held' = held$
$looprun' = looprun$
$clobbered' = clobbered$

`Ready` is the handshake write landing and the read loop starting.

Ready

ΔLeg
$s? : SESSION$

$looprun = \varnothing$
$phase\ s? = sattached$
$phase' = phase \oplus \{s? \mapsto spumping\}$
$leased' = leaseon$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$held' = held$
$looprun' = looprun$
$clobbered' = clobbered$

`Drop` is the loss of the socket — a peer that went away, a protocol violation, or a handshake write that failed — followed by the suspension this model turns on. The session is no longer serving anything, but it has removed nothing: it sits in `_pump`'s `finally`, awaiting the writer task it just cancelled, with all its installed state still in place.

Drop

ΔLeg
$s? : SESSION$

$looprun = \varnothing$
$phase\ s? \in \{sattached, spumping\}$
$phase' = phase \oplus \{s? \mapsto ssuspended\}$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$held' = held$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

7 The Teardown Path

Teardown resumes after the suspension and asks one question first: am I still the connection's listener? The answer decides everything, because the listener slot is the connection's ownership token. The comparison and the removals it authorises are one critical section, so no registration can land between the two.

`TeardownDetach` is the owned case. It clears the listener and, in the same step, the callbacks that listener owned; the ghost `clobbered` records a removal of anything installed by another session, which the guard is there to prevent. It also takes the loop's run token, so no reconnect can interleave before the disconnect cascade.

TeardownDetach

ΔLeg
$s? : SESSION$

$looprun = \varnothing$
$phase\ s? = ssuspended$
$listener = \{s?\}$
$phase' = phase \oplus \{s? \mapsto stearing\}$
$listener' = \varnothing$
$owner' = \varnothing$
$incumbent' = incumbent \setminus \{s?\}$
$looprun' = \{s?\}$
$clobbered' =$
  if $listener \cup \operatorname{ran} owner \subseteq \{s?\}$
  then $clobbered$ **else** $clobbered \cup \{s?\}$
$writer' = writer$
$gated' = gated$
$held' = held$
$leased' = leased$

`TeardownStale` is the other case: the session is no longer the listener. Two different situations reach it, and conflating them is a defect of its own. Either a successor has taken the connection, and the slot and its callbacks now belong to that successor; or the lease lapsed and the sweep took the session out from under a socket that was still winding down, and nobody holds the connection at all.

The slot and its callbacks are untouched either way — there is nothing here this session may remove. Its *own writer* is another matter, and it is released on this path too, guarded by identity: $writer \setminus \{s?\}$ removes the binding when it is still this session's and removes nothing when a successor has replaced it. Without that clause a swept session leaves its writer bound with nothing left in the system that could ever withdraw it (Section 13, variant 5).

TeardownStale

ΔLeg
$s? : SESSION$

$looprun = \varnothing$
$phase\ s? = ssuspended$
$listener \neq \{s?\}$
$phase' = phase \oplus \{s? \mapsto sdone\}$
$listener' = listener$
$writer' = writer \setminus \{s?\}$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$held' = held$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

`TeardownDisconnect` is the Hub's disconnect cascade — the pub-sub writer and the subscriptions — which runs after a successful detach, while the run token is still held. The removal is by the writer's own identity, $writer \setminus \{s?\}$, the same clause `TeardownStale` carries: the two paths differ in what they do to the slot, never in what they do to the session's own Hub-side state. The `clobbered` ghost is kept on this step even though the guard makes it unwritable, so that removing the guard is caught rather than merely allowed.

TeardownDisconnect

ΔLeg
$s? : SESSION$

$phase\ s? = stearing$
$looprun = \{s?\}$
$phase' = phase \oplus \{s? \mapsto sdone\}$
$writer' = writer \setminus \{s?\}$
$looprun' = \varnothing$
$clobbered' =$
  if $writer \subseteq \{s?\}$ **then** $clobbered$
  else $clobbered \cup \{s?\}$
$listener' = listener$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$held' = held$
$leased' = leased$

A finished session's slot is later occupied by a fresh process. In the implementation each reconnect constructs a new `HubListenSession`, so the reused element must not still be referenced anywhere; the guards say exactly that. `Reset` keeps the system live — it never runs out of enabled operations at a benign terminal state — so that the deadlock check reports only genuine wedges, never mere termination.

Reset

ΔLeg
$s? : SESSION$

$phase\ s? = sdone$
$looprun = \varnothing$
$s? \notin listener$
$s? \notin writer$
$s? \notin incumbent$
$s? \notin \mathrm{ran}\ owner$
$s? \notin \mathrm{ran}\ gated$
$phase' = phase \oplus \{s? \mapsto sfresh\}$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$held' = held$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

8 Registration

A registration arrives on another leg entirely — an MCP tool call or a REST request from the same identity, resolving to the same connection id — and therefore on another thread. It is two steps because the code makes it two: the push-reachability gate reads the router, and the commit writes the client registry.

`RegisterGate` is `has_listener`. It records which listener it saw, because that is the fact the commit must still be able to rely on.

RegisterGate

ΔLeg
$c? : CALLBACK$

$listener \neq \varnothing$
$leased = leaseon$
$c? \notin \mathrm{dom}\ owner$
$c? \notin \mathrm{dom}\ gated$
$gated' = gated \cup (\{c?\} \times listener)$
$phase' = phase$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$held' = held$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

`RegisterCommit` is the registry write. Its guard is the compare-and-set that the current code lacks: commit only if the listener that passed the gate is *still* the listener. Because the listener slot and the callback set are under one lock, the comparison and the write are one critical section, and no teardown can pass between them.

RegisterCommit

ΔLeg
$c? : CALLBACK$

$c? \in \mathrm{dom}\ gated$
$listener = \{gated\ c?\}$
$owner' = owner \oplus \{c? \mapsto gated\ c?\}$
$gated' = \{c?\} \lhd gated$
$phase' = phase$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$held' = held$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

`RegisterRefuse` is the same call losing the compare: the leg it was gated against has gone, so the caller is told it is not push-reachable rather than given an entry nothing can deliver.

RegisterRefuse

ΔLeg
$c? : CALLBACK$

$c? \in \mathrm{dom}\ gated$
$listener \neq \{gated\ c?\}$
$gated' = \{c?\} \lhd gated$
$phase' = phase$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$held' = held$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

9 Clicks, Lease, and Sweep

`Route` is a click on a registered callback landing in the connection's hold. It runs on the click-dispatch thread, so it is not subject to the run token.

Route

ΔLeg
$c? : CALLBACK$

$c? \in \mathrm{dom}\ owner$
$leased = leaseon$
$held' = held \cup \{c?\}$
$phase' = phase$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

`Drain` is the listener taking the hold and pushing its frames.

Drain

ΔLeg
$s? : SESSION$

$looprun = \varnothing$
$listener = \{s?\}$
$phase\ s? = spumping$
$held' = \varnothing$
$phase' = phase$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

Any frame from a pumping session renews the lease. This is the operation that makes a broken state permanent rather than self-healing: a session that is live and keeps renewing keeps the entry alive, so the sweep that would clear a wrongly-emptied entry never runs.

LeaseRenew

ΔLeg
$s? : SESSION$

$phase\ s? = spumping$
$leased' = leaseon$
$phase' = phase$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$held' = held$
$looprun' = looprun$
$clobbered' = clobbered$

LeaseLapse

ΔLeg

$leased = leaseon$
$\neg\ (\exists\, s : SESSION \bullet phase\ s \in \{sattached, spumping\})$
$leased' = leaseoff$
$phase' = phase$
$listener' = listener$
$writer' = writer$
$incumbent' = incumbent$
$owner' = owner$
$gated' = gated$
$held' = held$
$looprun' = looprun$
$clobbered' = clobbered$

`Sweep` is the opportunistic reap on a lapsed lease. Everything the registry holds for the session goes with the session record: its callbacks, its pending registrations, its hold, and its *listener slot*, because the slot is a field of the session object the reap deletes. A model that left the listener installed after a sweep would describe a state the implementation cannot occupy, and would then assert Invariant 1 over it. The `incumbent` ghost goes for the same reason: a session the registry has reaped is no longer one the design promises to keep push-reachable.

What does *not* go is the writer. It is bound in the Hub, a separate store the registry's reap cannot reach, so it outlives the sweep and only the session's own teardown can withdraw it. That asymmetry is real, and Invariant 5 is what holds the teardown to it.

Sweep

ΔLeg

$leased = leaseoff$
$owner' = \varnothing$
$gated' = \varnothing$
$held' = \varnothing$
$listener' = \varnothing$
$incumbent' = \varnothing$
$phase' = phase$
$writer' = writer$
$leased' = leased$
$looprun' = looprun$
$clobbered' = clobbered$

10 Invariants

Five properties must hold across all reachable states. None is placed in the `Leg` schema: a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant 1 — the incumbent stays reachable. The session whose connect most recently completed, and which has not itself detached, holds both the listener slot and the writer slot:

$$incumbent \neq \varnothing \Rightarrow (listener = incumbent \land writer = incumbent).$$

This is the safety core of "a live session is never left push-unreachable". It is what the stale-teardown clobber breaks.

Invariant 2 — callbacks belong to the current listener. Every registered callback was installed by the session that holds the listener slot now:

$$\mathrm{ran}\ owner \subseteq listener.$$

Since `# listener` $\leq$ `1`, this says both that no callback is registered without a listener and that none outlives the leg it was registered against. It is what the registration gap breaks.

Invariant 3 — no cross-session removal. No session ever removes state another session installed:

$$clobbered = \varnothing.$$

The ghost is written by exactly the two teardown steps that remove shared state, so its emptiness is a property of the transition relation expressed as a property of a state, and ProB reports the offending trace rather than merely the offending state.

Invariant 5 — a departed session leaves no writer behind. No session that has finished still holds the connection's writer:

$$\forall\, s : SESSION \bullet phase\, s = sdone \Rightarrow s \notin writer.$$

This is the Hub-side counterpart of Invariant 1. Invariant 1 says a live session stays reachable; Invariant 5 says a dead one stops being reachable, which is a different failure and needs its own statement. The writer is bound in the Hub, so no sweep of the registry can reclaim it: if a teardown declines to release it, it stays bound for the life of the process, and every publish on a topic it subscribed to goes on feeding a session that is gone. It is what a teardown that skips its own Hub-side state breaks (Section 13, variant 5).

Invariant 4 — deadlock freedom. No reachable state leaves the system unable to progress. The only mutual-exclusion device is `looprun`, taken by `TeardownDetach` and released by `TeardownDisconnect`; since `TeardownDisconnect`'s guard is implied by the state `TeardownDetach` establishes, the token cannot be held forever and no cyclic wait can form. ProB discharges this over the whole reachable state space.

11 Verification

Type-checking is a `fuzz` pass:

```
fuzz -t docs/listen_lifecycle.tex
```

Invariants 1–3 and 5 are state properties, checked by asking ProB whether their negation is *reachable*. A goal that is not found means the invariant holds on every reachable state:

```
# Invariant 1 (must report: goal NOT found)
probcli docs/listen_lifecycle.tex -model_check -nodead \
  -goal "incumbent /= {} & (listener /= incumbent or writer /= incumbent)" \
  -p DEFAULT_SETSIZE 2

# Invariant 2 (must report: goal NOT found)
probcli docs/listen_lifecycle.tex -model_check -nodead \
  -goal "not(ran(owner) <: listener)" \
  -p DEFAULT_SETSIZE 2

# Invariant 3 (must report: goal NOT found)
probcli docs/listen_lifecycle.tex -model_check -nodead \
  -goal "clobbered /= {}" \
  -p DEFAULT_SETSIZE 2

# Invariant 5 (must report: goal NOT found)
probcli docs/listen_lifecycle.tex -model_check -nodead \
  -goal "#ss.(ss : SESSION & phase(ss) = sdone & ss : writer)" \
  -p DEFAULT_SETSIZE 2
```

Invariant 4 is the deadlock check over the full reachable state space:

```
# Invariant 4 (must report: no deadlock, no invariant violation)
probcli docs/listen_lifecycle.tex -model_check \
  -p DEFAULT_SETSIZE 2
```

The last run also discharges the structural constraints of the `Leg` schema — in particular that `owner` and `gated` remain functional and that the occupancy slots stay singletons — since ProB carries a state schema's predicate as a machine invariant.

Results. At `DEFAULT_SETSIZE 2` the reachable space is 3 142 states over 14 563 transitions, every operation covered, no deadlock, and all four goals reported *not found*. At `DEFAULT_SETSIZE 3` — three sessions and three callbacks — it is 277 163 states over 1 885 221 transitions, again exhaustive, again with no counter-example to any goal. Two sessions are enough to exhibit every defect of Section 13; three confirm that nothing depends on the carrier being that small.

12 What the Model Asks of the Implementation

The verified design reduces to one idea: *the listener slot is the connection's ownership token, and every write to connection-bound state is a compare-and-set against it.* Three operations follow, and each must be a single critical section.

1. **Attach** (`Connect`). Install this session as the connection's listener and writer, and clear the connection's callbacks in the same step. A new occupant of the slot starts with no entries; the app re-registers from its `on_connect`.

2. **Register** (`RegisterGate` + `RegisterCommit`). Commit the callback only if the listener observed at the gate is still the listener. The comparison and the write must be one critical section — a re-read that is not atomic with the write reinstates the gap.

3. **Teardown** (`TeardownDetach` / `TeardownStale`). If this session is still the listener, clear the listener and the callbacks together; otherwise leave both alone. Either way release this session's *own* Hub-side state, by its own identity: the slot decides what belongs to whoever holds it, and the writer belongs to the session that bound it.

Those two removals answer different questions and must not be collapsed into one. "Am I still the listener?" decides the slot and its callbacks. "Is this writer still mine?" decides the Hub-side state, and it has a different answer in the case the sweep has already taken the session: nobody holds the slot, so the first question says leave everything, while the second correctly says take the writer. Reading the first answer as covering both is variant 5.

Two structural consequences follow, and neither is optional.

The listener slot and the callback set must live under *one* lock. They are in separate registries today — the slot in `CallbackRouter`, the callbacks on the session in `HubClientRegistry` — and the two locks are deliberately never nested, so as things stand no single critical section can span them. Moving the slot onto the session, and leaving the router with the holds alone, brings all three operations under the registry's existing lock. It adds no lock, nests nothing, and leaves the router's live-read-then-lock discipline untouched.

Ownership is decided by comparing the stored listener to `self`, and a generation counter adds nothing: a `HubListenSession` is constructed once per run of the `/ws` route and installs itself once, so its identity is already a monotone stamp. A counter would only restate that fact in a second place, where it could disagree. What would be unsound is a token that *can* recur — the connection id, or a bare "I installed something" flag — because a stale holder of such a token compares equal to the incumbent.

The writer slot needs no separate guard. It is removed by the disconnect cascade, which runs inside the same loop run as the detach that authorised it, so no reconnect can install a successor's writer in between; the model encodes this as the `looprun` token, and Invariant 1 holds with the writer removal unguarded. That conclusion depends on `_teardown` containing no `await`, which it must therefore continue not to.

13 Fidelity: reproducing the known defects

A model that cannot reproduce the defects it guards against proves nothing. Each variant below is the specification above with one named clause removed; each makes a different invariant reachable, and each

trace below is ProB's, not a hand-written one. Sessions are written A and B for `SESSION1` and `SESSION2`, callbacks c and d.

Variant 1 — the stale-teardown clobber

Delete the ownership guard `listener = {s?}` from `TeardownDetach`, and delete the `TeardownStale` schema (with no guard there is no stale case to divert). This is the current `_teardown`, which calls `remove_listener`, `withdraw_callbacks`, and `on_disconnect` on the connection id unconditionally.

The shortest trace to a live, push-unreachable session:

1. `Connect(A)`: A is the listener, the writer, and the incumbent.
2. `Drop(A)`: A's socket goes; A suspends in the `finally`, awaiting its cancelled writer. Nothing is removed yet.
3. `Connect(B)`, `Ready(B)`: the client reconnects after its backoff. B records, binds the writer, installs itself as the listener, becomes the incumbent, and starts pumping.
4. `TeardownDetach(A)`: A resumes and, unguarded, removes B's listener.
5. `TeardownDisconnect(A)`: A drops B's writer.

The resulting state has `incumbent = {B}` with `listener` and `writer` both empty — Invariant 1 — and `clobbered = {A}` — Invariant 3 — while B is `spumping` and `leased = leaseon`, so no sweep will ever repair it. ProB reaches the same violation by a second route in which A never drops at all, only is superseded; extending the goal to `owner = ∅` produces the thirteen-step trace in which B's registered callbacks are withdrawn by A's teardown as well.

Under the ownership guard, step 4 cannot fire: at that point `listener = {B}`, so A takes `TeardownStale` and removes nothing.

Variant 2 — the registration gap

Restore the guard of variant 1 and instead delete the compare-and-set `listener = {gated~c?}` from `RegisterCommit` (and, with it, `RegisterRefuse`, whose guard is its complement). This is the current `register_callback`, whose `has_listener` read and registry write are separate critical sections. The trace:

1. `Connect(A)`.
2. `Drop(A)`.
3. `RegisterGate(c)`: the gate reads `has_listener` as true — A has not torn down yet — and returns. The calling thread is now between the two locks.
4. `TeardownDetach(A)`: the listener goes and the callbacks are withdrawn.
5. `RegisterCommit(c)`: the write lands anyway.

The resulting state has `owner = {c ↦ A}` with `listener = ∅` — Invariant 2 — and the lease still live. The withdrawal that would have removed the entry has already run, so nothing will: a menu item that swallows clicks, which is exactly what the gate exists to prevent.

That this trace survives the ownership guard is the model's sharpest result. Guarding the teardown does not close the registration gap: with variant 1's guard restored and only this clause missing, ProB still reaches the violation in five steps, while reporting Invariant 1 unreachable over all 23 488 states. The two fixes are independent and both are required.

Variant 3 — the superseded registration

Restore both guards and instead replace `owner' = ∅` in `Connect` with `owner' = owner` — the current `add_listener`, which replaces the listener and leaves the connection's callbacks alone. Two sessions of one identity may be live at once, which the identity-derived connection id permits by design:

1. `Connect(A)`, `RegisterGate(c)`, `RegisterCommit(c)`.
2. `Connect(B)`: B replaces A in the listener slot.

Now `owner = {c ↦ A}` while `listener = {B}` — Invariant 2. A's menu entry is still in the bar and its clicks are routed to B. This is not a race: it is reachable with no interleaving at all, and it is why the fixed design makes installing a listener clear the connection's callbacks.

Variant 4 — the split detach

Restore everything and instead split `TeardownDetach` into its two writes — `TeardownRemoveListener` then `TeardownWithdraw` — as `_teardown` has them today, keeping the ownership guard on the first and the run token across both. This asks whether the two calls may stay separate once the other three clauses are in place. They may not:

1. `Connect(A)`, `Connect(B)`, `RegisterGate(c)`, `RegisterCommit(c)`: B holds the slot and owns c.
2. `Drop(B)`.
3. `TeardownRemoveListener(B)`: the listener is gone; the callbacks are not.

That intermediate state has `owner = {c ↦ B}` with `listener = ∅` — Invariant 2 — and it is observable, because `route` and `register` run on other threads and may read the registry inside the window. The window is transient, so this is a narrower fault than variant 2's permanent orphan; but it costs nothing to close, because the recommended fix already places the listener slot and the callback set under one lock, where clearing both is one operation.

Variant 5 — the teardown that skips its own Hub-side state

Restore everything and instead replace `writer' = writer \ {s?}` in `TeardownStale` with `writer' = writer`. This is the teardown that treats "I am not the listener" as "none of this is mine", which reads correctly for the slot and is wrong for the writer. The trace needs no successor at all:

1. `Connect(A)`: A is the listener, the writer, and the incumbent.
2. `Drop(A)`: the socket goes; A suspends in the `finally`, still holding everything.
3. `LeaseLapse`, `Sweep`: the lease lapses while the socket winds down and the reap takes the session record — with it the listener slot, the callbacks, and the incumbent ghost. The writer is in the Hub, so it stays.
4. `TeardownStale(A)`: A resumes, finds it is not the listener, and removes nothing.

The resulting state has `phase A = sdone` with `writer = {A}` — Invariant 5. Nothing can repair it: the registry has already forgotten the session, so no later sweep reaches the Hub, and the entry that could have been overwritten by a reconnect never is if the process is gone for good. In the implementation this is the dead session's `deliver_event` left subscribed, filling an outbound queue whose write task was cancelled.

The trace is short and needs no pathology: a client may declare a lease as short as five seconds while the keepalive cadence is fifteen, so the lapse in step 3 is ordinary operation, not a stall. Invariants 1–3 stay unreachable throughout, which is the point — the ownership guard on the slot does not imply the release of the writer, and each has to be stated.

Each of the five variants is therefore necessary, and together they are sufficient: with all five clauses in place the model is exhaustively clean.